CAD 图纸转换标准操作流程（SOP）

> 版本: v1.0
>
> 日期: 2026-04-08
>
> 依据: SVG*HOTZONE*SPEC v1.0 第五节
>
> 适用范围: 将物业 CAD (.dwg) 楼层图纸转换为可交互 SVG + 热区映射文件

一、流程总览

```
.dwg 原始图纸
    │
    ▼  [Step 1] ODA File Converter
.dxf 中间格式
    │
    ▼  [Step 2] Python ezdxf 脚本
.svg 线稿（无热区）
    │
    ▼  [Step 3] 热区标注（Inkscape 手工 + 脚本辅助）
.svg 含热区标记
    │
    ▼  [Step 4] 自动提取 floor_map.json
.json 映射文件
    │
    ▼  [Step 5] 校验 & 上传
后端存储 → 前端渲染
```

二、环境准备

2.1 工具清单

工具	版本要求	用途	安装方式
ODA File Converter	≥ 25.3	DWG → DXF 格式转换	官网下载（免费）
Python	≥ 3.10	运行转换脚本	`brew install python@3.12` (macOS)
ezdxf	≥ 1.1	DXF 解析与 SVG 生成	`pip install ezdxf[draw]`
svgwrite	≥ 1.4	SVG 文件操作	`pip install svgwrite` （ezdxf draw 组件已包含）
lxml	≥ 4.9	SVG XML 解析与编辑	`pip install lxml`
Inkscape	≥ 1.3	SVG 可视化编辑、热区手工标注	`brew install --cask inkscape` (macOS)

2.2 Python 虚拟环境

```
# 在项目根目录
cd /Users/wanxt/app/propos
python3 -m venv .venv
source .venv/bin/activate
pip install ezdxf[draw] lxml svgwrite
```

2.3 目录结构

```
propos/
├── cad_source/                    # 原始 CAD 文件（不入版本控制）
│   ├── building_a/
│   │   ├── 1F.dwg
│   │   ├── 2F.dwg
│   │   └── ...
│   └── building_b/
├── cad_intermediate/              # 中间文件（DXF、未标注 SVG）
│   └── ...
├── floors/                        # 最终输出（与后端 FILE_STORAGE_PATH 对应）
│   ├── {building_id}/
│   │   ├── {floor_id}.svg
│   │   └── {floor_id}.json
│   └── ...
└── scripts/
    ├── cad_to_dxf.sh            # Step 1 批量转换脚本
    ├── dxf_to_svg.py            # Step 2 DXF→SVG 转换
    ├── annotate_hotzone.py      # Step 3 热区辅助标注
    └── extract_floor_map.py     # Step 4 JSON 提取
```

三、Step 1： DWG → DXF 格式转换

3.1 使用 ODA File Converter

ODA File Converter 提供命令行批量转换模式：

```
# macOS 路径（安装后）
ODA_CONVERTER="/Applications/ODAFileConverter.app/Contents/MacOS/ODAFileConverter"

# 参数说明：
# 参数1：输入目录
# 参数2：输出目录
# 参数3：输出版本（ACAD2018 兼容性最好）
# 参数4：输出格式（DXF）
# 参数5：递归处理（0=否，1=是）
# 参数6：审计模式（1=启用，修复损坏图形）

"$ODA_CONVERTER" \
  "cad_source/building_a" \
  "cad_intermediate/building_a" \
  "ACAD2018" "DXF" "0" "1"
```

3.2 批量转换脚本

```
#!/bin/bash
# scripts/cad_to_dxf.sh
# 用法：bash scripts/cad_to_dxf.sh <building_folder>
# 示例：bash scripts/cad_to_dxf.sh building_a

set -euo pipefail

BUILDING="$1"
INPUT_DIR="cad_source/${BUILDING}"
OUTPUT_DIR="cad_intermediate/${BUILDING}"
ODA="/Applications/ODAFileConverter.app/Contents/MacOS/ODAFileConverter"

if [[ ! -d "$INPUT_DIR" ]]; then
  echo "错误：输入目录 $INPUT_DIR 不存在"
  exit 1
fi

mkdir -p "$OUTPUT_DIR"

echo "=== 开始转换 ${BUILDING} ==="
echo "输入：$INPUT_DIR"
echo "输出：$OUTPUT_DIR"

"$ODA" "$INPUT_DIR" "$OUTPUT_DIR" "ACAD2018" "DXF" "0" "1"

echo "=== 转换完成 ==="
```

```
ls -la "$OUTPUT_DIR"/*.dxf 2>/dev/null | wc -l | xargs -I {} echo "共转换 {} 个 DXF 文件"
```

3.3 质量检查

转换完成后检查：

- [ ] 输出文件数 = 输入文件数
- [ ] 每个 DXF 文件 > 0KB（排除空文件）
- [ ] 用 Inkscape 或 AutoCAD Viewer 抽样打开 DXF 验证线稿完整

四、Step 2： DXF → SVG 转换

4.1 转换脚本

```
#!/usr/bin/env python3
"""
scripts/dxf_to_svg.py
DXF → SVG 自动转换脚本

用法：
  python scripts/dxf_to_svg.py <input.dxf> <output.svg> [--width 1200]
  python scripts/dxf_to_svg.py cad_intermediate/building_a/1F.dxf cad_intermediate/building_a/
1F.svg
"""

import argparse
import sys
from pathlib import Path

import ezdxf
from ezdxf.addons.drawing import RenderContext, Frontend
from ezdxf.addons.drawing.matplotlib import MatplotlibBackend
from ezdxf.addons.drawing.svg import SVGBackend


def convert_dxf_to_svg(input_path: str, output_path: str, target_width: int = 1200) -> None:
    """将 DXF 文件转换为 SVG。

    Args:
        input_path: DXF 文件路径
        output_path: 输出 SVG 文件路径
        target_width: 目标视口宽度（像素）
    """
    doc = ezdxf.readfile(input_path)
    msp = doc.modelspace()

    # 创建渲染上下文
    ctx = RenderContext(doc)
    backend = SVGBackend()
```

```
    frontend = Frontend(ctx, backend)

    # 渲染所有可见图层
    frontend.draw_layout(msp)

    # 写入 SVG
    svg_string = backend.get_string(
        # 保持宽高比，设定宽度
    )

    output = Path(output_path)
    output.parent.mkdir(parents=True, exist_ok=True)
    output.write_text(svg_string, encoding="utf-8")
    print(f"✅ 转换完成: {input_path} → {output_path}")


def main():
    parser = argparse.ArgumentParser(description="DXF to SVG converter")
    parser.add_argument("input", help="输入 DXF 文件路径")
    parser.add_argument("output", help="输出 SVG 文件路径")
    parser.add_argument("--width", type=int, default=1200, help="目标 SVG 宽度（像素）")
    args = parser.parse_args()

    if not Path(args.input).exists():
        print(f"错误: 输入文件不存在 - {args.input}", file=sys.stderr)
        sys.exit(1)

    convert_dxf_to_svg(args.input, args.output, args.width)


if __name__ == "__main__":
    main()
```

4.2 批量转换

```
# 遍历某栋楼所有 DXF 文件并转换
for dxf in cad_intermediate/building_a/*.dxf; do
  svg="${dxf%.dxf}.svg"
  python scripts/dxf_to_svg.py "$dxf" "$svg"
done
```

4.3 SVG 后处理

自动转换的 SVG 需要进行以下后处理：

1. **注入标准样式块** — 添加 SVG*HOTZONE*SPEC 规定的 `<defs><style>` 样式
2. **分层整理** — 将 CAD 线稿元素包裹到 `<g id="floor-plan">` 中
3. **清理冗余** — 移除 CAD 图框、标题栏、图层中不需要的辅助线

```
#!/usr/bin/env python3
"""
```

```
scripts/postprocess_svg.py
SVG 后处理：注入标准样式 + 分层整理

用法：
  python scripts/postprocess_svg.py <input.svg> <output.svg>
"""

from lxml import etree
from pathlib import Path
import sys

SVG_NS = "http://www.w3.org/2000/svg"
NSMAP = {None: SVG_NS}

STANDARD_STYLES = """
      /* 状态色块 — 运行时由前端根据 unit.current_status 动态切换 class */
      .unit-leased       { fill: #4CAF50; fill-opacity: 0.35; stroke: #388E3C; stroke-width: 1;
}
      .unit-vacant        { fill: #F44336; fill-opacity: 0.35; stroke: #D32F2F; stroke-width:
1; }
      .unit-expiring-soon { fill: #FF9800; fill-opacity: 0.35; stroke: #F57C00; stroke-width:
1; }
      .unit-renovating    { fill: #2196F3; fill-opacity: 0.35; stroke: #1976D2; stroke-width:
1; }
      .unit-non-leasable  { fill: #9E9E9E; fill-opacity: 0.20; stroke: #757575; stroke-width:
1; }
      /* hover 效果 */
      [data-unit-id]:hover { fill-opacity: 0.55; cursor: pointer; }
"""


def postprocess(input_path: str, output_path: str) -> None:
    tree = etree.parse(input_path)
    root = tree.getroot()

    # 1. 注入 <defs><style>（如果不存在）
    defs = root.find(f"{{{SVG_NS}}}defs")
    if defs is None:
        defs = etree.SubElement(root, f"{{{SVG_NS}}}defs")
        root.insert(0, defs)

    existing_style = defs.find(f"{{{SVG_NS}}}style")
    if existing_style is None:
        style_el = etree.SubElement(defs, f"{{{SVG_NS}}}style")
        style_el.text = STANDARD_STYLES

    # 2. 将所有现有内容包裹到 <g id="floor-plan">
    floor_plan = root.find(f".//{{{SVG_NS}}}g[@id='floor-plan']")
    if floor_plan is None:
        floor_plan = etree.SubElement(root, f"{{{SVG_NS}}}g")
        floor_plan.set("id", "floor-plan")
        floor_plan.set("pointer-events", "none")
```

```
        # 移动非 defs 的子元素到 floor-plan
        for child in list(root):
            if child.tag != f"{{{SVG_NS}}}defs" and child is not floor_plan:
                floor_plan.append(child)

    # 3. 创建空的热区层
    hotspots = root.find(f".//{{{SVG_NS}}}g[@id='unit-hotspots']")
    if hotspots is None:
        hotspots = etree.SubElement(root, f"{{{SVG_NS}}}g")
        hotspots.set("id", "unit-hotspots")

    # 写入
    Path(output_path).parent.mkdir(parents=True, exist_ok=True)
    tree.write(output_path, xml_declaration=True, encoding="utf-8", pretty_print=True)
    print(f"✅ 后处理完成: {output_path}")


if __name__ == "__main__":
    if len(sys.argv) != 3:
        print("用法: python scripts/postprocess_svg.py <input.svg> <output.svg>",
file=sys.stderr)
        sys.exit(1)
    postprocess(sys.argv[1], sys.argv[2])
```

五、Step 3：热区标注

5.1 标注方式选择

方式	适用场景	效率
Inkscape 手工标注	非规则形状、首次标注	低（~15min/层）
脚本辅助标注	规则矩形单元、批量标注	高（~2min/层）
半自动（推荐）	混合场景：脚本处理矩形，手工调整异形	中

5.2 Inkscape 手工标注步骤

1. **打开已后处理的 SVG**

```bash
inkscape cad*intermediate/building*a/1F_processed.svg
```

1. **选中热区层**

图层面板 → 选择 `unit-hotspots` 图层（如无则创建）

1. **绘制单元区域**

- 矩形单元：使用矩形工具（R）绘制覆盖该单元的矩形
- 异形单元：使用贝塞尔工具（B）描边围出区域
- 多边形单元：使用多边形工具描点

1. 添加自定义属性

选中刚绘制的形状 → XML 编辑器（Ctrl+Shift+X）→ 添加：

- `data-unit-id`：从数据库查询该单元的 UUID
- `data-unit-number`：单元编号（如 `101`）
- 修改 `class` 为 `unit-vacant`

1. 保存

文件 → 保存为 → 普通 SVG（非 Inkscape SVG）

5.3 脚本辅助标注

针对规则排列的单元（如公寓标间），可用脚本批量生成热区：

```
#!/usr/bin/env python3
"""
scripts/annotate_hotzone.py
半自动热区标注脚本 — 根据 CSV 配置为规则排列的单元生成矩形热区

输入 CSV 格式 (units_layout.csv):
unit_id,unit_number,x,y,width,height
c0000000-...,101,120,50,200,150
c0000000-...,102,330,50,200,150

用法:
  python scripts/annotate_hotzone.py <input.svg> <layout.csv> <output.svg>
"""

import csv
import sys
from lxml import etree
from pathlib import Path

SVG_NS = "http://www.w3.org/2000/svg"


def annotate(svg_path: str, csv_path: str, output_path: str) -> None:
    tree = etree.parse(svg_path)
    root = tree.getroot()

    # 找到或创建热区层
    hotspots = root.find(f".//{{{SVG_NS}}}g[@id='unit-hotspots']")
    if hotspots is None:
        hotspots = etree.SubElement(root, f"{{{SVG_NS}}}g")
        hotspots.set("id", "unit-hotspots")

    # 读取 CSV 并生成矩形
    count = 0
```

```
    with open(csv_path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            rect = etree.SubElement(hotspots, f"{{{SVG_NS}}}rect")
            rect.set("data-unit-id", row["unit_id"])
            rect.set("data-unit-number", row["unit_number"])
            rect.set("class", "unit-vacant")
            rect.set("x", row["x"])
            rect.set("y", row["y"])
            rect.set("width", row["width"])
            rect.set("height", row["height"])
            count += 1

    Path(output_path).parent.mkdir(parents=True, exist_ok=True)
    tree.write(output_path, xml_declaration=True, encoding="utf-8", pretty_print=True)
    print(f"✅ 已标注 {count} 个单元热区 → {output_path}")


if __name__ == "__main__":
    if len(sys.argv) != 4:
        print("用法: python scripts/annotate_hotzone.py <input.svg> <layout.csv> <output.svg>",
              file=sys.stderr)
        sys.exit(1)
    annotate(sys.argv[1], sys.argv[2], sys.argv[3])
```

5.4 单元 UUID 查询

标注时需要查询数据库获取 `unit_id`：

```
-- 查询某楼层所有单元的 UUID 和编号
SELECT u.id AS unit_id, u.unit_number, b.name AS building, f.floor_number
FROM units u
JOIN floors f ON u.floor_id = f.id
JOIN buildings b ON f.building_id = b.id
WHERE b.name = 'A座' AND f.floor_number = 1
ORDER BY u.unit_number;
```

可导出为 CSV 后作为 `annotate_hotzone.py` 的输入。

六、Step 4：提取 floor_map.json

6.1 提取脚本

```
#!/usr/bin/env python3
"""
scripts/extract_floor_map.py
从已标注的 SVG 提取 floor_map.json

用法:
```

```
  python scripts/extract_floor_map.py <input.svg> <building_id> <floor_id> [--output
<output.json>]
"""

import argparse
import json
import re
import sys
from lxml import etree
from pathlib import Path

SVG_NS = "http://www.w3.org/2000/svg"


def parse_points(points_str: str) -> list[list[float]]:
    """解析 SVG polygon points 属性为坐标数组。"""
    pairs = points_str.strip().split()
    result = []
    for pair in pairs:
        parts = pair.split(",")
        if len(parts) == 2:
            result.append([float(parts[0]), float(parts[1])])
    return result


def compute_centroid_rect(x: float, y: float, w: float, h: float) -> dict:
    return {"x": round(x + w / 2, 1), "y": round(y + h / 2, 1)}


def compute_centroid_polygon(points: list[list[float]]) -> dict:
    n = len(points)
    if n == 0:
        return {"x": 0, "y": 0}
    cx = sum(p[0] for p in points) / n
    cy = sum(p[1] for p in points) / n
    return {"x": round(cx, 1), "y": round(cy, 1)}


def extract(svg_path: str, building_id: str, floor_id: str) -> dict:
    tree = etree.parse(svg_path)
    root = tree.getroot()

    # 获取视口尺寸
    viewbox = root.get("viewBox", "0 0 1200 800")
    parts = viewbox.split()
    viewport = {
        "width": int(float(parts[2])) if len(parts) >= 3 else 1200,
        "height": int(float(parts[3])) if len(parts) >= 4 else 800,
    }

    units = []
    # 查找所有带 data-unit-id 的元素
    for elem in root.iter():
```

```
        unit_id = elem.get("data-unit-id")
        if not unit_id:
            continue

        unit_number = elem.get("data-unit-number", "")
        tag = etree.QName(elem.tag).localname

        unit_entry = {
            "unit_id": unit_id,
            "unit_number": unit_number,
        }

        if tag == "rect":
            x = float(elem.get("x", 0))
            y = float(elem.get("y", 0))
            w = float(elem.get("width", 0))
            h = float(elem.get("height", 0))
            unit_entry["shape"] = "rect"
            unit_entry["bounds"] = {"x": x, "y": y, "width": w, "height": h}
            unit_entry["label_position"] = compute_centroid_rect(x, y, w, h)

        elif tag == "polygon":
            points = parse_points(elem.get("points", ""))
            unit_entry["shape"] = "polygon"
            unit_entry["points"] = points
            unit_entry["label_position"] = compute_centroid_polygon(points)

        elif tag == "path":
            path_d = elem.get("d", "")
            unit_entry["shape"] = "path"
            unit_entry["path_d"] = path_d
            # 简单近似 — 从 path 中提取所有坐标点计算质心
            nums = re.findall(r"[-+]?\d*\.?\d+", path_d)
            if len(nums) >= 2:
                coords = list(zip(
                    [float(nums[i]) for i in range(0, len(nums), 2)],
                    [float(nums[i]) for i in range(1, len(nums), 2)],
                ))
                unit_entry["label_position"] = compute_centroid_polygon(
                    [[c[0], c[1]] for c in coords]
                )
            else:
                unit_entry["label_position"] = {"x": 0, "y": 0}

        units.append(unit_entry)

    return {
        "floor_id": floor_id,
        "building_id": building_id,
        "svg_version": Path(svg_path).stat().st_mtime.__int__().__str__()[:10],
        "viewport": viewport,
        "units": units,
    }
```

```
def main():
    parser = argparse.ArgumentParser(description="从 SVG 提取 floor_map.json")
    parser.add_argument("svg", help="已标注的 SVG 文件路径")
    parser.add_argument("building_id", help="楼栋 UUID")
    parser.add_argument("floor_id", help="楼层 UUID")
    parser.add_argument("--output", "-o", help="输出 JSON 路径（默认与 SVG 同名）")
    args = parser.parse_args()

    if not Path(args.svg).exists():
        print(f"错误: SVG 文件不存在 - {args.svg}", file=sys.stderr)
        sys.exit(1)

    result = extract(args.svg, args.building_id, args.floor_id)

    output_path = args.output or args.svg.replace(".svg", ".json")
    Path(output_path).parent.mkdir(parents=True, exist_ok=True)
    with open(output_path, "w", encoding="utf-8") as f:
        json.dump(result, f, ensure_ascii=False, indent=2)

    print(f"✅ 已提取 {len(result['units'])} 个单元 → {output_path}")


if __name__ == "__main__":
    main()
```

七、Step 5：校验与上传

7.1 校验清单

#	校验项	方法	通过标准
1	SVG 可正常渲染	浏览器打开 SVG 文件	线稿完整，无黑屏
2	热区可视化正确	浏览器打开，hover 热区高亮	每个标注的单元都能高亮
3	data-unit-id 唯一	脚本检查	SVG 内无重复 unit_id
4	unit_id 存在于数据库	SQL 交叉查询	所有标注的 UUID 在 units 表中存在
5	楼层归属正确	SQL 查询 units.floor_id	所有标注单元属于该楼层
6	JSON 与 SVG 同步	对比 JSON units 数组与 SVG 热区元素数量	数量一致，unit_id 集合相同
7	单元数量完整	对比 JSON units 数组与数据库该楼层单元数	可标注单元全覆盖

7.2 校验脚本

```
#!/bin/bash
```

```
# scripts/validate_floor_svg.sh
# 用法: bash scripts/validate_floor_svg.sh <floor.svg> <floor.json>

set -euo pipefail

SVG_FILE="$1"
JSON_FILE="$2"

echo "=== 校验 SVG: $SVG_FILE ==="

# 检查文件存在
[[ -f "$SVG_FILE" ]] || { echo "❌ SVG 文件不存在"; exit 1; }
[[ -f "$JSON_FILE" ]] || { echo "❌ JSON 文件不存在"; exit 1; }

# 提取 SVG 中 unit_id 数量
SVG_COUNT=$(grep -o 'data-unit-id=' "$SVG_FILE" | wc -l | tr -d ' ')
echo "SVG 热区数量: $SVG_COUNT"

# 提取 JSON 中 unit 数量
JSON_COUNT=$(python3 -c "
import json
with open('$JSON_FILE') as f:
    data = json.load(f)
print(len(data.get('units', [])))
")
echo "JSON 单元数量: $JSON_COUNT"

# 比较
if [[ "$SVG_COUNT" == "$JSON_COUNT" ]]; then
  echo "✅ SVG 与 JSON 单元数量一致: $SVG_COUNT"
else
  echo "❌ 数量不一致! SVG=$SVG_COUNT, JSON=$JSON_COUNT"
  exit 1
fi

# 检查 SVG 中有无重复 unit_id
DUPLICATE_COUNT=$(grep -oP 'data-unit-id="\K[^"]+' "$SVG_FILE" | sort | uniq -d | wc -l | tr -d
' ')
if [[ "$DUPLICATE_COUNT" -gt 0 ]]; then
  echo "❌ 发现 $DUPLICATE_COUNT 个重复 unit_id:"
  grep -oP 'data-unit-id="\K[^"]+' "$SVG_FILE" | sort | uniq -d
  exit 1
else
  echo "✅ 无重复 unit_id"
fi

echo "=== 校验通过 ==="
```

7.3 上传到后端

```
# 复制到文件存储目录
BUILDING_ID="a0000000-0000-0000-0000-000000000001"
FLOOR_ID="b0000000-0000-0000-0000-000000000001"

DEST_DIR="${FILE_STORAGE_PATH}/floors/${BUILDING_ID}"
mkdir -p "$DEST_DIR"
cp final_1F.svg  "$DEST_DIR/${FLOOR_ID}.svg"
cp final_1F.json "$DEST_DIR/${FLOOR_ID}.json"

# 或通过 API 上传
curl -X POST "https://api.propos.example.com/api/floors/${FLOOR_ID}/upload-plan" \
  -H "Authorization: Bearer $TOKEN" \
  -F "svg=@final_1F.svg" \
  -F "json=@final_1F.json"
```

八、完整操作示例

以 A座 1楼 为例，完整走一遍 5 步流程：

```
# === Step 1: DWG → DXF ===
bash scripts/cad_to_dxf.sh building_a
# 输出: cad_intermediate/building_a/1F.dxf

# === Step 2: DXF → SVG ===
source .venv/bin/activate
python scripts/dxf_to_svg.py \
  cad_intermediate/building_a/1F.dxf \
  cad_intermediate/building_a/1F_raw.svg

# SVG 后处理
python scripts/postprocess_svg.py \
  cad_intermediate/building_a/1F_raw.svg \
  cad_intermediate/building_a/1F_processed.svg

# === Step 3: 热区标注 ===
# 方式A: 规则布局 — 先查数据库导出 CSV
psql $DATABASE_URL -c "
  COPY (
    SELECT u.id AS unit_id, u.unit_number, 0 AS x, 0 AS y, 200 AS width, 150 AS height
    FROM units u
    JOIN floors f ON u.floor_id = f.id
    JOIN buildings b ON f.building_id = b.id
    WHERE b.name = 'A座' AND f.floor_number = 1
    ORDER BY u.unit_number
  ) TO STDOUT WITH CSV HEADER
" > cad_intermediate/building_a/1F_layout.csv
# 手动编辑 CSV 中的 x,y,width,height 坐标值（参照 SVG 中的位置）
```

```
python scripts/annotate_hotzone.py \
  cad_intermediate/building_a/1F_processed.svg \
  cad_intermediate/building_a/1F_layout.csv \
  cad_intermediate/building_a/1F_annotated.svg

# 方式B: 异形单元— Inkscape 手工编辑
inkscape cad_intermediate/building_a/1F_annotated.svg

# === Step 4: 提取 JSON ===
python scripts/extract_floor_map.py \
  cad_intermediate/building_a/1F_annotated.svg \
  a0000000-0000-0000-0000-000000000001 \
  b0000000-0000-0000-0000-000000000001 \
  -o floors/a0000000-0000-0000-0000-000000000001/b0000000-0000-0000-0000-000000000001.json

# 同步 SVG 到最终目录
cp cad_intermediate/building_a/1F_annotated.svg \
   floors/a0000000-0000-0000-0000-000000000001/b0000000-0000-0000-0000-000000000001.svg

# === Step 5: 校验 ===
bash scripts/validate_floor_svg.sh \
  floors/a0000000-0000-0000-0000-000000000001/b0000000-0000-0000-0000-000000000001.svg \
  floors/a0000000-0000-0000-0000-000000000001/b0000000-0000-0000-0000-000000000001.json
```

九、常见问题排查

问题	可能原因	解决方案
DXF 无输出 / 文件为空	DWG 版本过高	ODA 输出版本改为 `ACAD2013`
SVG 线稿缺失	DXF 使用了外部参照（XREF）	进入 CAD 执行 `BIND` 绑定外部参照后重新导出
SVG 全黑 / 太大	viewBox 范围不正确	手动调整 SVG `viewBox` 或使用 ezdxf 限定渲染范围
热区位置偏移	SVG 坐标系原点与 CAD 不一致	检查 DXF 中是否有 UCS 偏移，或在 SVG 中使用 `transform` 修正
JSON 提取数量不对	热区元素不在 `unit-hotspots` 层中	确保所有热区元素都在 `<g id="unit-hotspots">` 下
Inkscape 保存后属性丢失	保存为 Inkscape SVG 会丢弃自定义属性	保存为「普通 SVG」或「优化的 SVG」

十、维护与更新

当楼层装修导致单元分合变更时：

1. **数据库先行**: 在 `units` 表中完成单元增删改
2. **更新 SVG**: 修改对应楼层 SVG 中的热区形状和 `data-unit-id`
3. **重新提取 JSON**: 重新运行 `extract_floor_map.py`
4. **校验**: 执行 `validate_floor_svg.sh`
5. **上传覆盖**: 替换原文件，前端下次加载自动生效

> 变更记录应在 `renovation_records` 表中留痕，对应 SVG 版本号更新 `floor_map.json` 中的 `svg_version` 字段。